

The Decision Tree Learning Process 🌲

The algorithm for building a decision tree is a **recursive process**. It starts with the entire dataset at the top (the root) and progressively splits the data into smaller and smaller subsets.

The core of the process is a loop that repeats at each node:

1. Decide on the **best feature** to split the current set of examples.
 2. **Split** the data into new subsets (branches) based on that feature's values.
 3. **Repeat** this process on each new subset.
 4. **Stop** when a criterion is met, at which point a **leaf node** (a final prediction) is created.
-

A Step-by-Step Example

Using our 10-example (5 cats, 5 dogs) dataset:

1. Start at the Root Node:

- We begin with all **10 examples**.
- The algorithm must choose the best feature to split on. Let's say it determines "**Ear Shape**" is the best.
- It splits the 10 examples into two branches:
 - **Left Branch:** 5 examples with "Pointy" ears (4 cats, 1 dog).
 - **Right Branch:** 5 examples with "Floppy" ears (1 cat, 4 dogs).

2. Recurse on the Left Branch:

- We now focus *only* on the 5 "Pointy" examples.
- The algorithm again chooses the best feature for *this subset*. Let's say it picks "**Face Shape**".
- It splits these 5 examples:
 - **Left-Left Branch:** 4 examples with "Round" face.
 - **Left-Right Branch:** 1 example with "Not Round" face.

3. Create Leaf Nodes (Stop Splitting):

- The "Left-Left Branch" contains 4 examples, and all are "Cat". This node is **100% pure**. We stop here and create a **leaf node** with the prediction: "**Cat**".
- The "Left-Right Branch" contains 1 example, which is "Not Cat". This node is also **100% pure**. We create a **leaf node**: "**Not Cat**".

4. Recurse on the Right Branch:

- We now return to the 5 "Floppy" examples.

- The algorithm chooses the best feature for this subset. Let's say it picks "**Whiskers**".
- It splits these 5 examples:
 - **Right-Left Branch:** 1 example with "Present" whiskers.
 - **Right-Right Branch:** 4 examples with "Absent" whiskers.

5. Create More Leaf Nodes:

- The "Right-Left Branch" has 1 example, which is "Cat". It's pure. We create a **leaf node: "Cat"**.
- The "Right-Right Branch" has 4 examples, all of which are "Not Cat". It's pure. We create a **leaf node: "Not Cat"**.

The tree is now fully built.

Key Decisions in Tree Building

The algorithm must make two critical decisions at each step:

1. How to Choose the Best Feature to Split On?

- **Goal:** The algorithm seeks to maximize **purity** (or minimize **impurity**). A node is "pure" if all examples in it belong to the same class (e.g., 100% cats).
- **Process:** The algorithm will test every possible feature (Ear Shape, Face Shape, Whiskers) and see which split results in the "purest" child nodes.
- **Example:** A hypothetical "Cat DNA" feature would be perfect, as it would create two 100% pure groups immediately. Without it, the algorithm picks the next best thing.
- **Metric:** This measure of purity/impurity is formally calculated using a metric called **entropy**, which will be covered in the next video.

2. When to Stop Splitting?

If we continue splitting until every node is 100% pure, the tree will become very large and complex, leading to **overfitting**. To prevent this, we use **stopping criteria**:

1. **Node Purity:** Stop when a node is 100% pure (all examples are the same class).
 2. **Maximum Depth:** Stop splitting if a node is at a certain "depth" (e.g., 10 hops from the root). This is a hyperparameter you can set.
 3. **Minimum Purity Gain:** Stop if a split only provides a tiny, insignificant improvement in purity (below a certain threshold).
 4. **Minimum Examples:** Stop if a node has fewer than a certain number of examples (e.g., 5 examples). Splitting a tiny group is often just fitting noise in the data.
-

Conclusion: A "Messy" but Effective Algorithm

The decision tree algorithm may seem like a collection of many different rules and heuristics. This is because it evolved over decades, with different researchers adding new refinements (like different stopping criteria). While it might feel "messy" compared to the clean mathematics of logistic regression, these pieces fit together to create a very effective and widely used algorithm.